there is no way *built into* the language to do I/O in CoffeeScript (or JavaScript). What do you use instead? You use callback functions.

[2] Excluding a brief encounter with fs.readFile in listing 2.4.

3.3. I/O

Continuing the scenic route to the power of functions, you're presented with a new problem: there's no easy, built-in way to do I/O in CoffeeScript (or JavaScript). What can you do? The strategy so far for parts of the language you want but that are not built in has been to define a functions for them. It's the same with I/O; you define a function that specifies how to handle data resulting from I/O.

This method of doing I/O seems tedious at first but has two advantages. First, it means that I/O is asynchronous (something you'll learn much more about in chapter 9), and second, it means you don't have to look at the details of how the I/O is implemented. Instead, you concentrate on the callback function. But before you can ignore a detail, you need to appreciate what it is you're ignoring. The I/O details for fetching data in a web browser are commonly grouped under the term *Ajax*.

3.3.1. Ajax

Remember that party attendees website? You want the count to update every second, but so far you have no way to actually get the attendee count. Suppose the attendee count is available somewhere on the web; it doesn't matter where. To get that data into the website you use Ajax to fetch it. Time for a bit of wishful thinking: suppose you already have a `get` function that does the Ajax for you and that this `get` function accepts a callback function as an argument:

```
url = "http://www.coffeescriptinaction.com/3/data.js"
get url, (response) ->
  console.log response
```

Use the get function to dispatch a request.

Figure 3.3 demonstrates how this works across the network.